# CDAC FUTURE SKILLS PROGRAMME

## Demystifying Android Application Security

## Day 02 – Assignment: Android Static Analysis

JADX, APKTool and MobSF – Standard Practical Tasks


**Name** : Vimal Mudalagi

**Institution** : Digital University Kerala

**Programme** : M.Sc. Computer Science (Specialization: Cyber Security)

## Task 0 – APK Identification

| Item | Value |
|---|---|
| Filename | `InsecureShop.apk` |
| File size | 4,754,534 bytes (~4.54 MB) |
| SHA-256 | `a83298ae4a37fcab8101e8b41e513dd2199af71a94ea537d556a318e07d4d1bd` |
| Analysis date | 23-09-2026 |

# Part A – Manual Static Analysis (JADX and APKTool)

## Task 1 – JADX: APK & Class Identification

**Objective:** To decompile the provided APK using JADX-GUI and identify its package name, main activity, and one security-relevant class.

| Item | Observation |
|---|---|
| APK | `InsecureShop.apk` |
| Package name | `com.insecureshop` |
| Main/Launcher Activity | `com.insecureshop.ProductListActivity` |
| Security-relevant class | `com.insecureshop.LoginActivity` |
| Tool | JADX-GUI |

JADX-GUI was used to decompile **InsecureShop.apk** and inspect its package structure and source code. The package name was identified as `com.insecureshop`. `ProductListActivity` was identified as the main launcher activity from the **MAIN** and **LAUNCHER** intent-filter. `LoginActivity` was selected as a security-relevant class because it handles user authentication/login functionality.

**Screenshot 1**

*JADX manifest view showing the InsecureShop package name and launcher activity.*

File View Navigation Tools Plugins Help *InsecureShop - jadx-gui

- InsecureShop.apk
  - Inputs
    - Files
      - InsecureShop.apk
  - Source code
    - android.support.v4
    - androidx
    - com
    - kotlin
    - kotlinx.android
    - net.gotev.uploadservice
    - org
  - Resources
    - kotlin
    - META-INF
    - res
    - AndroidManifest.xml
    - classes.dex
    - resources.arsc
  - APK signature
  - Summary

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="1"
    android:versionName="1.0"
    android:compileSdkVersion="29"
    android:compileSdkVersionCodename="10"
    package="com.insecureshop"
    platformBuildVersionCode="29"
    platformBuildVersionName="10">
    <uses-sdk
        android:minSdkVersion="16"
        android:targetSdkVersion="29"/>
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.READ_CONTACTS"/>
    <permission android:name="com.insecureshop.permission.READ"/>
    <uses-permission android:name="android.permission.WAKE_LOCK"/>
    <application
        android:theme="@style/AppTheme"
        android:label="@string/app_name"
        android:icon="@mipmap/ic_launcher"
        android:name="com.insecureshop.InsecureShopApp"
        android:debuggable="true"
        android:allowBackup="true"
        android:supportsRtl="true"
        android:usesCleartextTraffic="true"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:appComponentFactory="androidx.core.app.CoreComponentFactory">
        <activity
            android:name="com.insecureshop.ChooserActivity"
            android:excludeFromRecents="true">
            <intent-filter>
                <action android:name="android.intent.action.VIEW"/>
                <category android:name="android.intent.category.DEFAULT"/>
            </intent-filter>
            <intent-filter>
                <action android:name="android.intent.action.SEND"/>
                <category android:name="android.intent.category.DEFAULT"/>
                <data android:mimeType="application/*"/>
                <data android:mimeType="audio/*"/>
                <data android:mimeType="image/*"/>
                <data android:mimeType="text/*"/>
                <data android:mimeType="video/*"/>
            </intent-filter>
            <meta-data
                android:name="android.service.chooser.chooser_target_service"
                android:value=".ConversationChooserTargetService"/>
        </activity>
        <activity
            android:name="com.insecureshop.AboutUsActivity"
```

**Screenshot 2**

*JADX decompiled source showing the security-relevant LoginActivity class.*

File View Navigation Tools Plugins Help *InsecureShop - jadx-gui

- InsecureShop.apk
  - Inputs
    - Files
      - InsecureShop.apk
  - Source code
    - android.support.v4
    - androidx
    - com
      - bumptech.glide
      - google
      - insecureshop
        - broadcast
        - contentProvider
        - databinding
        - util
        - AboutUsActivity
        - BR
        - BuildConfig
        - Cart
        - CartAdapter
        - CartListActivity
        - ChooserActivity
        - CustomReceiver
        - DataBinderMapperImpl
        - DataBindingInfo
        - InsecureShopApp
        - LoginActivity
          - _$_findViewCache HashMap
          - mBinding ActivityLoginBinding
          - LoginActivity() void
          - _$_clearFindViewByIdCache() v
          - _$_findCachedViewById(int) Vi
          - getMBinding() ActivityLoginBi
          - onCreate(Bundle) void
          - onLogin(View) void
          - setMBinding(ActivityLoginBind
        - PrivateActivity
        - ProductAdapter
        - ProductDetail
        - ProductListActivity
        - R
        - ResultActivity
        - SendingDataViaActionActivity
        - WebView2Activity
        - WebViewActivity
    - kotlin
    - kotlinx.android

AndroidManifest.xml | LoginActivity

Find: LoginActivity

```
import android.content.Context;
import android.content.Intent;
import android.content.pm.PackageInfo;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.databinding.DataBindingUtil;
import androidx.databinding.ViewDataBinding;
import com.google.android.material.textfield.TextInputEditText;
import com.insecureshop.databinding.ActivityLoginBinding;
import com.insecureshop.util.Prefs;
import com.insecureshop.util.Util;
import java.util.HashMap;
import kotlin.Metadata;
import kotlin.jvm.internal.Intrinsics;
import kotlin.text.StringsKt;

/* JADX INFO: compiled from: LoginActivity.kt */
/* JADX INFO: loaded from: classes.dex */
@Metadata(bv = {1, 0, 3}, d1 = {"\u0000(\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u
public final class LoginActivity extends AppCompatActivity {
    private HashMap _$_findViewCache;
    public ActivityLoginBinding mBinding;

    public void _$_clearFindViewByIdCache() {
        HashMap map = this._$_findViewCache;
        if (map != null) {
            map.clear();
        }
    }

    public View _$_findCachedViewById(int i) {
        if (this._$_findViewCache == null) {
            this._$_findViewCache = new HashMap();
        }
        View view = (View) this._$_findViewCache.get(Integer.valueOf(i));
        if (view != null) {
            return view;
        }
        View viewFindViewById = findViewById(i);
        this._$_findViewCache.put(Integer.valueOf(i), viewFindViewById);
        return viewFindViewById;
    }
```

## Task 2 – Keyword Search in Decompiled Source

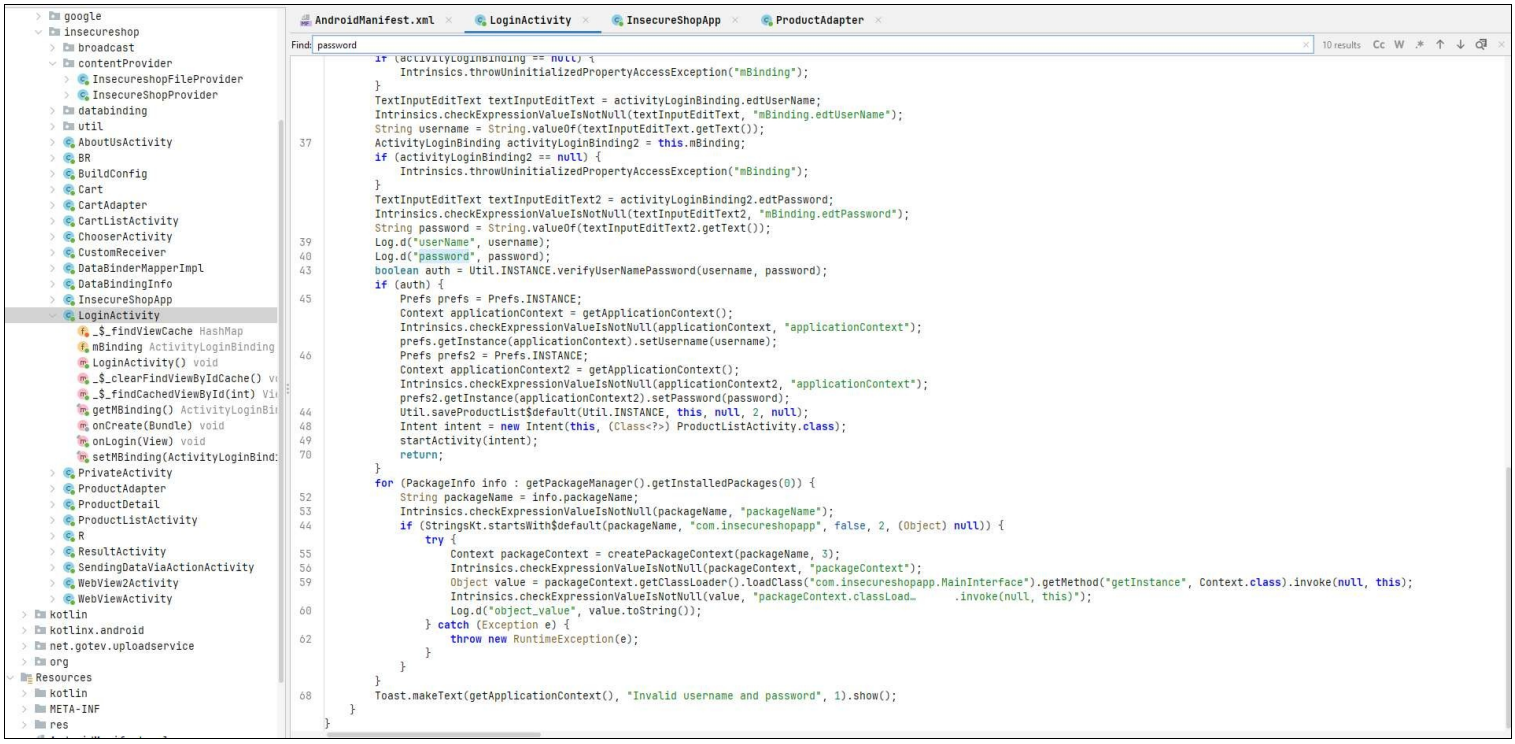
**Objective:** To search the decompiled project for security-relevant keywords using JADX (Ctrl+Shift+F) and identify genuinely relevant results.

### Result 1: LoginActivity

Keyword search for **password** in `LoginActivity` located the `onLogin()` method, which retrieves the password from the input field, passes it to the authentication function, and also writes it to the Android debug log using `Log.d("password", password)`. The password is subsequently passed to `setPassword(password)`. These operations are security-relevant because they involve handling and logging/storing authentication credentials.

### Screenshot 1

*LoginActivity opened in JADX with the onLogin() method visible.*

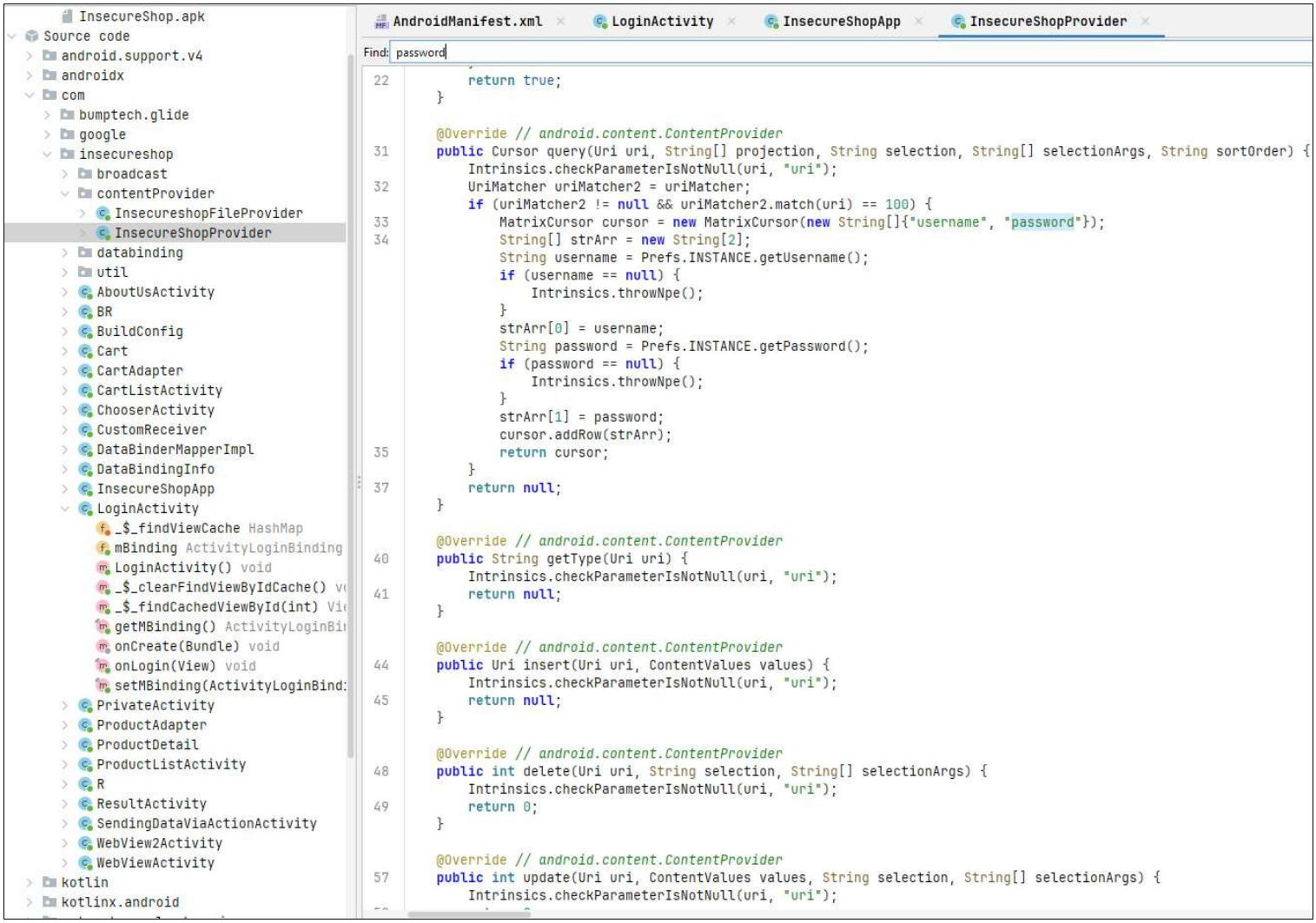



### Result 2: InsecureShopProvider

Keyword search for **password** in `InsecureShopProvider` located the `query()` method, which retrieves the stored username and password via `Prefs` and places both values into a returned `MatrixCursor`. This is security-relevant because the class is a ContentProvider handling and returning credential data.

### Screenshot 2

*InsecureShopProvider opened in JADX with the query() method visible.*




### Task 2 – Keyword Search Table

| Keyword | Class | Method | Security-relevant observation |
|---|---|---|---|
| `password` | `LoginActivity` | `onLogin()` | Password is retrieved, logged with Log.d(), and passed to setPassword(). |
| `password` | `InsecureShopProvider` | `query()` | Stored username and password are retrieved and placed into a returned Cursor. |

## Task 3 – APKTool Decoding

**Objective:** To decode the APK using APKTool and inspect the resulting Android application structure.

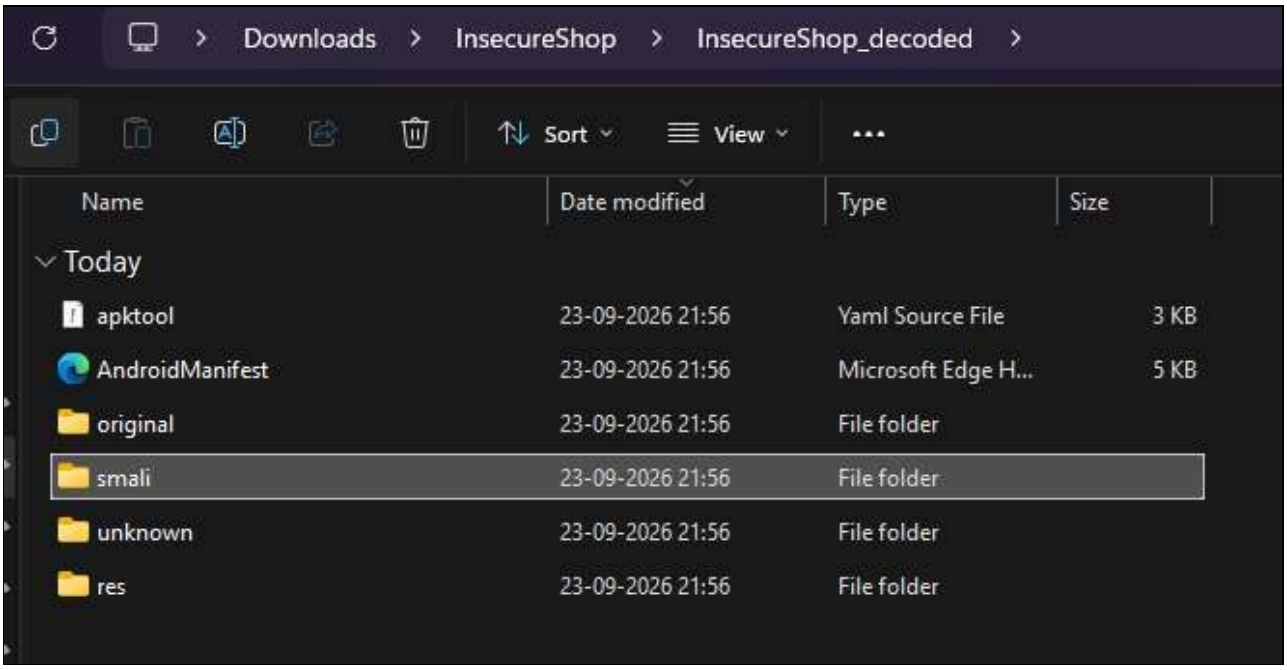
**Command:**

```
.\apktool.bat d InsecureShop.apk -o InsecureShop_decoded
```

**Observation:** APKTool 3.0.3 successfully decoded `InsecureShop.apk`. The decoded application directory was created as `InsecureShop_decoded`. The APKTool output confirmed decoding of the resource table, `classes.dex`, resources, and `AndroidManifest.xml`. The decoded folder contained **AndroidManifest.xml**, **res/**, and **smali/** as expected.

**Screenshot 1**

*File Explorer showing the InsecureShop_decoded folder containing AndroidManifest.xml, res, and smali.*




**Screenshot 2**

*PowerShell terminal showing the apktool decode command and the directory listing before/after decoding.*

```
PS C:\Users\KUDSIT\Downloads\InsecureShop> dir


    Directory: C:\Users\KUDSIT\Downloads\InsecureShop


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        23-09-2026     21:55             48 apktool.bat
-a----        23-09-2026     21:53       15478013 apktool_3.0.3.jar
-a----        23-09-2026     21:12        4754534 InsecureShop.apk


PS C:\Users\KUDSIT\Downloads\InsecureShop> .\apktool.bat d InsecureShop.apk -o InsecureShop_decoded
I: Using Apktool 3.0.3 on InsecureShop.apk with 8 threads
I: Loading resource table...
I: Baksmaling classes.dex...
I: Decoding value resources...
I: Loading resource table from file: C:\Users\KUDSIT\AppData\Local\apktool\framework\1.apk
I: Decoding file resources...
I: Generating values XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Copying original files...
I: Copying unknown files...
PS C:\Users\KUDSIT\Downloads\InsecureShop> dir


    Directory: C:\Users\KUDSIT\Downloads\InsecureShop


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        23-09-2026     21:56                InsecureShop_decoded
-a----        23-09-2026     21:55             48 apktool.bat
-a----        23-09-2026     21:53       15478013 apktool_3.0.3.jar
-a----        23-09-2026     21:12        4754534 InsecureShop.apk
```

## Task 4 – AndroidManifest.xml Security Review

**Objective:** To inspect the decoded AndroidManifest.xml for security-relevant application settings, exported components, and permissions.

### 1. Security Flags

| Setting | Value | Observation |
|---|---|---|
| `android:debuggable` | true | Debugging is enabled |
| `android:allowBackup` | true | Application data backup is allowed |
| `android:usesCleartextTraffic` | true | Cleartext HTTP traffic is permitted |
| `android:networkSecurityConfig` | Not observed | No custom network security configuration declared |

These configurations were recorded as **security-relevant configurations** and were not treated as confirmed vulnerabilities at this stage.

### 2. Exported Components

| Component | Exported | Observation |
|---|---|---|
| `ChooserActivity` | Not explicitly declared | Has intent filters |
| `AboutUsActivity` | true | Explicitly exported |
| `CartListActivity` | Not explicitly declared | No exported attribute |
| `ProductListActivity` | Not explicitly declared | Has MAIN/LAUNCHER |
| `LoginActivity` | Not explicitly declared | No exported attribute |
| `WebViewActivity` | Not explicitly declared | Has VIEW/BROWSABLE intent filter |
| `WebView2Activity` | Not explicitly declared | Has BROWSABLE intent filter |
| `PrivateActivity` | false | Explicitly non-exported |
| `SendingDataViaActionActivity` | Not explicitly declared | No exported attribute |
| `ResultActivity` | true | Explicitly exported |
| `InsecureShopProvider` | true | Exported ContentProvider |
| `FileProvider` | false | Non-exported |
| `UploadService` | true | Exported service |

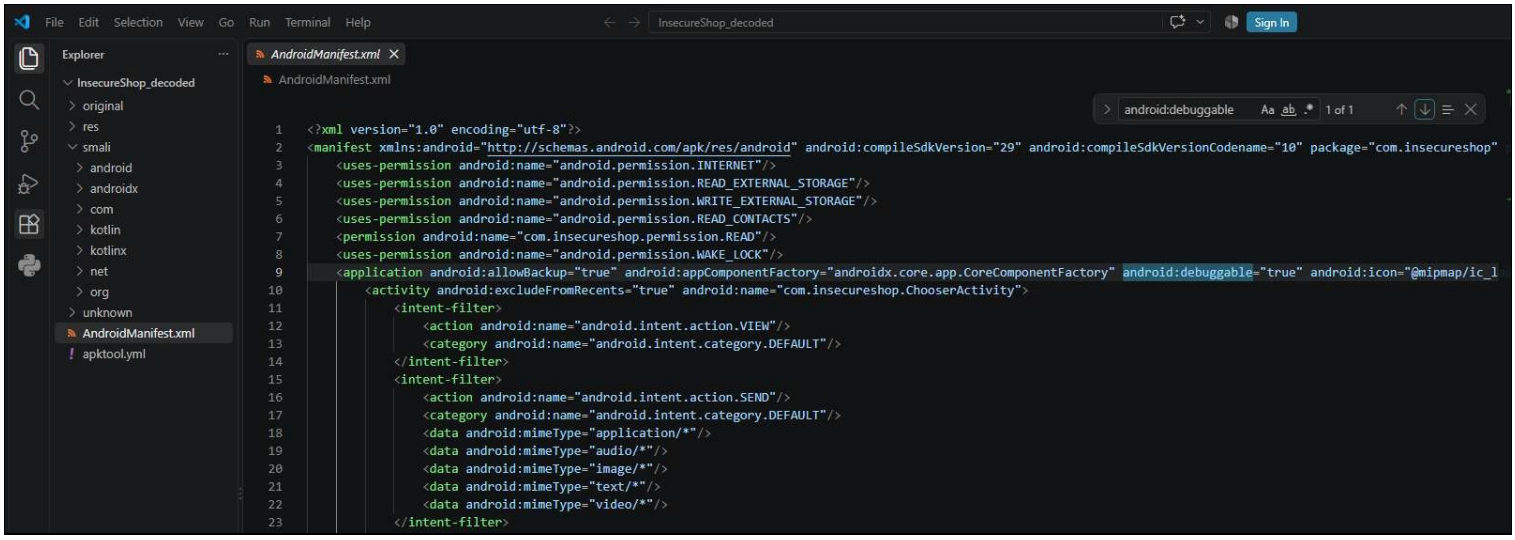
For components without an explicit `android:exported` attribute, this was not simply recorded as **false** – the effective behaviour can depend on Android version and intent-filter/component rules.

### 3. Permissions

The manifest declares: `android.permission.INTERNET`, `android.permission.READ_EXTERNAL_STORAGE`, `android.permission.WRITE_EXTERNAL_STORAGE`, `android.permission.READ_CONTACTS`, `android.permission.WAKE_LOCK`, and a custom `com.insecureshop.permission.READ` permission, which is also used as the `android:readPermission` on the exported provider.

**Screenshot 1**

*Decoded AndroidManifest.xml with android:allowBackup highlighted.*




**Screenshot 2**

*Decoded AndroidManifest.xml showing android:debuggable and other application-level flags.*

**Screenshot 3**

*Decoded AndroidManifest.xml with android:usesCleartextTraffic highlighted.*

**Screenshot 4**

*Decoded AndroidManifest.xml, further view confirming the usesCleartextTraffic setting.*

AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android" android:compileSdkVersion="29" android:compileSdkVersionCodename="10"
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.READ_CONTACTS"/>
    <permission android:name="com.insecureshop.permission.READ"/>
    <uses-permission android:name="android.permission.WAKE_LOCK"/>
    <application android:allowBackup="true" android:appComponentFactory="androidx.core.app.CoreComponentFactory"
    android:debuggable="true" android:icon="@mipmap/ic_launcher" android:label="@string/app_name"
    android:name="com.insecureshop.InsecureShopApp" android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true" android:theme="@style/AppTheme" android:usesCleartextTraffic="true">
        <activity android:excludeFromRecents="true" android:name="com.insecureshop.ChooserActivity">
            <intent-filter>
                <action android:name="android.intent.action.VIEW"/>
                <category android:name="android.intent.category.DEFAULT"/>
            </intent-filter>
```

## Task 5 – Deep Links and WebView Configuration

**Objective:** To review the manifest and JADX source for custom URI schemes/deep links and risky WebView configuration.

| Item | Observation |
|---|---|
| Scheme | `insecureshop` |
| Host | `com.insecureshop` |
| Component | `WebViewActivity` |
| WebView component | `WebView2Activity` |

### WebView findings

| Configuration | Observation |
|---|---|
| `setJavaScriptEnabled(true)` | JavaScript enabled |
| `setAllowUniversalAccessFromFileURLs(true)` | Universal access from file URLs enabled |
| `loadUrl()` | URL can be obtained from Intent data/query parameter/extras |

**Observation:** The application defines a custom `insecureshop://` URI scheme. `WebView2Activity` enables JavaScript and universal access from file URLs. It also passes URL values obtained from incoming Intent data, query parameters, or extras directly to `WebView.loadUrl()`. These findings are security-relevant and require further validation; they are not treated as confirmed vulnerabilities based solely on static analysis.

#### Screenshot

*WebView2Activity source in JADX showing setJavaScriptEnabled, setAllowUniversalAccessFromFileURLs, and loadUrl() reading data from the incoming Intent.*

Find: loadUrl

```
12          return this.USER_AGENT;
        }

        @Override // androidx.appcompat.app.AppCompatActivity, androidx.fragment.app.FragmentActivity, androidx.act
15      protected void onCreate(Bundle savedInstanceState) {
16          super.onCreate(savedInstanceState);
17          setContentView(R.layout.activity_webview);
18          setSupportActionBar((Toolbar) _$_findCachedViewById(R.id.toolbar));
19          setTitle(getString(R.string.webview));
21          Intent extraIntent = (Intent) getIntent().getParcelableExtra("extra_intent");
22          if (extraIntent != null) {
23              startActivity(extraIntent);
24              finish();
25              return;
            }
28          WebView webview = (WebView) findViewById(R.id.webview);
30          Intrinsics.checkExpressionValueIsNotNull(webview, "webview");
            WebSettings settings = webview.getSettings();
            Intrinsics.checkExpressionValueIsNotNull(settings, "webview.settings");
            boolean z = true;
            settings.setJavaScriptEnabled(true);
31          WebSettings settings2 = webview.getSettings();
            Intrinsics.checkExpressionValueIsNotNull(settings2, "webview.settings");
            settings2.setLoadWithOverviewMode(true);
32          WebSettings settings3 = webview.getSettings();
            Intrinsics.checkExpressionValueIsNotNull(settings3, "webview.settings");
            settings3.setUseWideViewPort(true);
33          WebSettings settings4 = webview.getSettings();
            Intrinsics.checkExpressionValueIsNotNull(settings4, "webview.settings");
            settings4.setAllowUniversalAccessFromFileURLs(true);
34          WebSettings settings5 = webview.getSettings();
            Intrinsics.checkExpressionValueIsNotNull(settings5, "webview.settings");
            settings5.setUserAgentString(this.USER_AGENT);
35          Intent intent = getIntent();
            Intrinsics.checkExpressionValueIsNotNull(intent, "intent");
            String dataString = intent.getDataString();
            if (!(dataString == null || StringsKt.isBlank(dataString))) {
36              Intent intent2 = getIntent();
                Intrinsics.checkExpressionValueIsNotNull(intent2, "intent");
                webview.loadUrl(intent2.getDataString());
42              return;
            }
37          Intent intent3 = getIntent();
            Intrinsics.checkExpressionValueIsNotNull(intent3, "intent");
            Uri data = intent3.getData();
            String queryParameter = data != null ? data.getQueryParameter("url") : null;
            if (!(queryParameter == null || StringsKt.isBlank(queryParameter))) {
```

## Task 6 – Resources and Assets Review

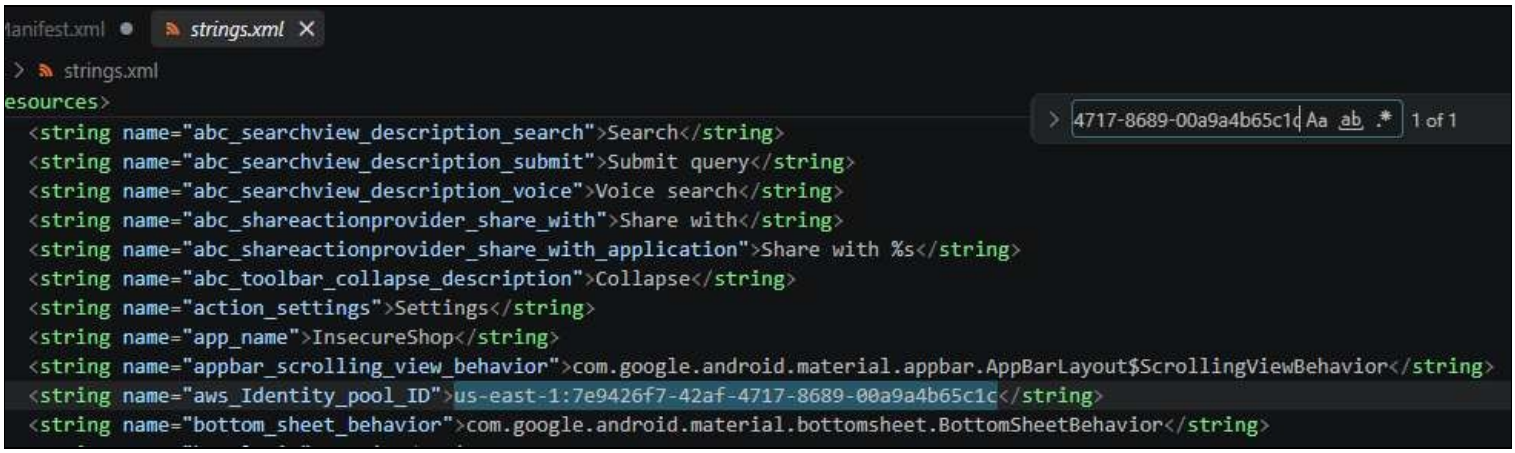
**Objective:** To review decoded resource and asset files for hardcoded sensitive values such as passwords, tokens, secrets, API keys, server URLs, or encryption keys.

The decoded application resources were reviewed for passwords, tokens, secrets, API keys, encryption keys, and server URLs. In `res/values/strings.xml`, an AWS Cognito Identity Pool ID was observed: `us-east-1:7e9426f7-42af-4717-8689-00a9a4b65c1c`. This value is an identifier and was not treated as a confirmed secret. The `res/xml`, `res/raw`, and `assets` directories were not present in the decoded application. No confirmed passwords, authentication tokens, or encryption keys were identified during this resource review.

| Location | Finding | Assessment |
|---|---|---|
| `res/values/strings.xml` | AWS Cognito Identity Pool ID | Identifier; not treated as a confirmed secret |
| `res/xml` | Not present | No matching output observed |
| `res/raw` | Not present | No matching output observed |
| `assets` | Not present | No matching output observed |

**Screenshot**

*strings.xml showing the hardcoded AWS Cognito Identity Pool ID.*



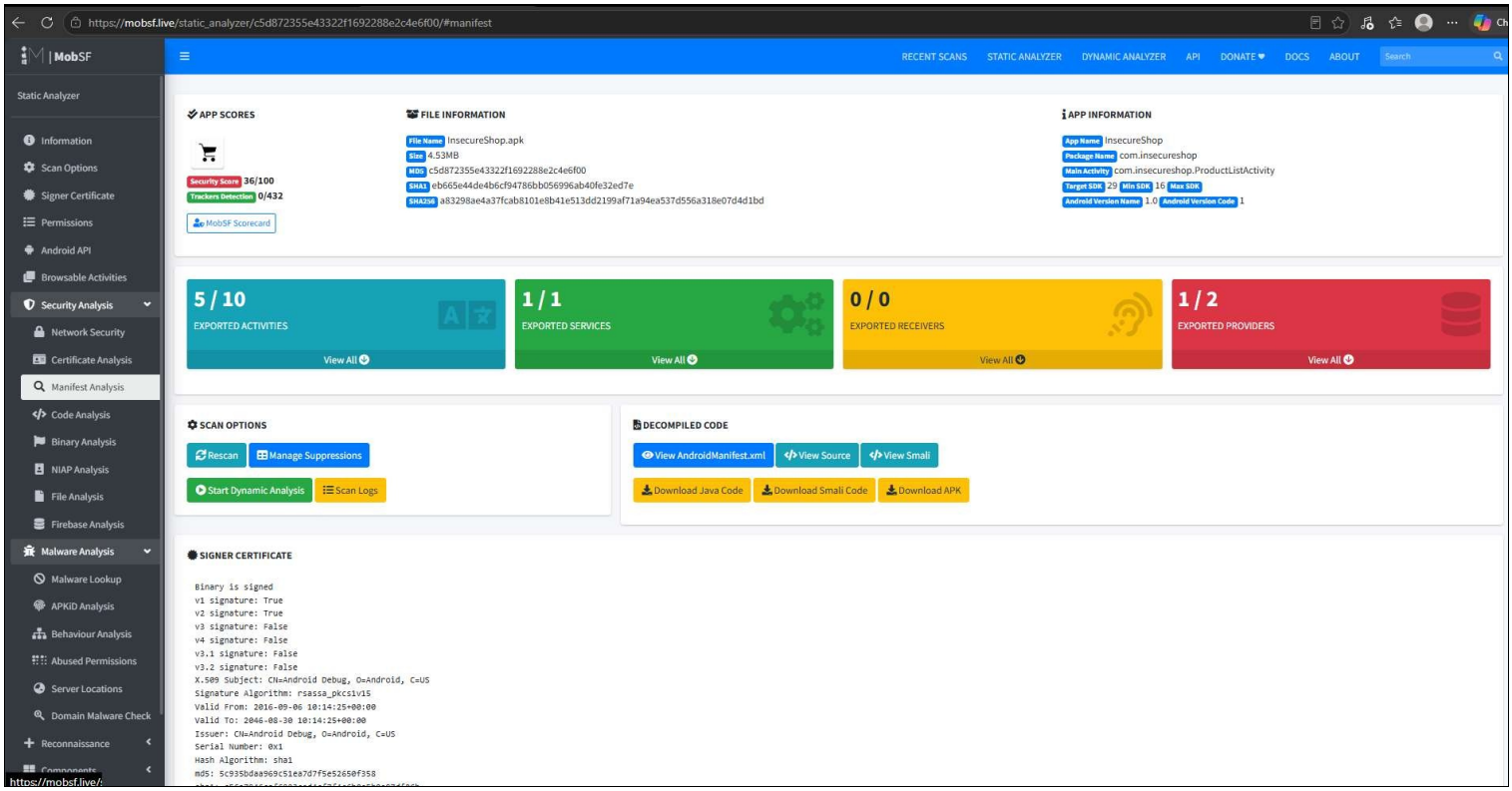
# Part B – Automated Cross-Check (MobSF)

## Task 8 – MobSF Scan

**Objective:** To scan the target APK using MobSF and record the overall security score along with one manifest-related and one code-related finding.

### Screenshot 1

*MobSF application overview showing the overall security score and file/app information.*




### Manifest finding

| MobSF Finding | Severity | Evidence |
|---|---|---|
| Clear text traffic is enabled (`android:usesCleartextTraffic=true`) | High | `AndroidManifest.xml` |

### Screenshot 2

*MobSF Manifest Analysis table identifying cleartext traffic enabled.*

MANIFEST ANALYSIS

| HIGH | WARNING | INFO | SUPPRESSED |
|---|---|---|---|
| 3 | 8 | 0 | 0 |

Search:

| NO | ISSUE | SEVERITY | DESCRIPTION | OPTIONS |
|---|---|---|---|---|
| 1 | App can be installed on a vulnerable unpatched Android version 4.1-4.1.2, [minSdk=16] | high | This application can be installed on an older version of android that has multiple unfixed vulnerabilities. These devices won't receive reasonable security updates from Google. Support an Android version => 10, API 29 to receive reasonable security updates. | |
| 2 | Clear text traffic is Enabled For App[android:usesCleartextTraffic=true] | high | The app intends to use cleartext network traffic, such as cleartext HTTP, FTP stacks, DownloadManager, and MediaPlayer. The default value for apps that target API level 27 or lower is "true". Apps that target API level 28 or higher default to "false". The key reason for avoiding cleartext traffic is the lack of confidentiality, authenticity, and protections against tampering; a network attacker can eavesdrop on transmitted data and also modify it without being detected. | |
| 3 | Debug Enabled For App [android:debuggable=true] | high | Debugging was enabled on the app which makes it easier for reverse engineers to hook a debugger to it. This allows dumping a stack trace and accessing debugging helper classes. | |
| 4 | Application Data can be Backed up [android:allowBackup=true] | warning | This flag allows anyone to backup your application data via adb. It allows users who have enabled USB debugging to copy application data off of the device. | |
| 5 | Activity (com.insecureshop.ChooserActivity) is not Protected. An intent-filter exists. | warning | An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Activity is explicitly exported. | |
| 6 | Activity (com.insecureshop.AboutUsActivity) is not Protected. [android:exported=true] | warning | An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. | |
| 7 | Activity (com.insecureshop.WebViewActivity) is not Protected. An intent-filter exists. | warning | An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Activity is explicitly exported. | |
| 8 | Activity (com.insecureshop.WebView2Activity) is not Protected. An intent-filter exists. | warning | An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Activity is explicitly exported. | |
| 9 | Activity (com.insecureshop.ResultActivity) is not Protected. [android:exported=true] | warning | An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. | |
| 10 | Content Provider (com.insecureshop.contentProvider.InsecureShopProvider) is not Protected. [android:exported=true] | warning | A Content Provider is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. | |

Showing 1 to 10 of 11 entries

## Code finding

| MobSF Finding | Severity | File |
|---|---|---|
| Insecure WebView implementation; SSL certificate errors are ignored and any SSL certificate is accepted | High | `com/insecureshop/util/CustomWebViewClient.java` |

### Screenshot 3

*MobSF Code Analysis table identifying insecure SSL certificate validation in CustomWebViewClient.java.*

CODE ANALYSIS

| HIGH | WARNING | INFO | SECURE | SUPPRESSED |
|---|---|---|---|---|
| 2 | 3 | 2 | 0 | 0 |

Search:

| NO | ISSUE | SEVERITY | STANDARDS | FILES | OPTIONS |
|---|---|---|---|---|---|
| 1 | The App logs information. Sensitive information should never be logged. | info | **CWE:** CWE-532: Insertion of Sensitive Information into Log File<br>**OWASP MASVS:** MSTG-STORAGE-3 | Show Files | |
| 2 | Ensure that user controlled URLs never reaches the Webview. Enabling file access from URLs in WebView can leak sensitive information from the file system. | warning | **CWE:** CWE-200: Information Exposure<br>**OWASP Top 10:** M1: Improper Platform Usage<br>**OWASP MASVS:** MSTG-PLATFORM-7 | com/insecureshop/PrivateActivity.java<br>com/insecureshop/WebView2Activity.java<br>com/insecureshop/WebViewActivity.java | |
| 3 | Insecure WebView Implementation. WebView ignores SSL Certificate errors and accept any SSL Certificate. This application is vulnerable to MITM attacks | high | **CWE:** CWE-295: Improper Certificate Validation<br>**OWASP Top 10:** M3: Insecure Communication<br>**OWASP MASVS:** MSTG-NETWORK-3 | com/insecureshop/util/CustomWebViewClient.java | |
| 4 | App can write to App Directory. Sensitive Information should be encrypted. | info | **CWE:** CWE-276: Incorrect Default Permissions<br>**OWASP MASVS:** MSTG-STORAGE-14 | com/insecureshop/util/Prefs.java | |
| 5 | App can read/write to External Storage. Any App can read data written to External Storage. | warning | **CWE:** CWE-276: Incorrect Default Permissions<br>**OWASP Top 10:** M2: Insecure Data Storage<br>**OWASP MASVS:** MSTG-STORAGE-2 | com/insecureshop/ChooserActivity.java | |
| 6 | Files may contain hardcoded sensitive information like usernames, passwords, keys etc. | warning | **CWE:** CWE-312: Cleartext Storage of Sensitive Information<br>**OWASP Top 10:** M9: Reverse Engineering<br>**OWASP MASVS:** MSTG-STORAGE-14 | Show Files | |
| 7 | Debug configuration enabled. Production builds must not be debuggable. | high | **CWE:** CWE-919: Weaknesses in Mobile Applications<br>**OWASP Top 10:** M1: Improper Platform Usage<br>**OWASP MASVS:** MSTG-RESILIENCE-2 | com/insecureshop/BuildConfig.java | |

Showing 1 to 7 of 7 entries

## Task 8 – Summary

| Item | Result |
|---|---|
| Security Score | **36/100** |
| Manifest finding | Cleartext traffic enabled (`android:usesCleartextTraffic=true`) — High |
| Code finding | Insecure WebView SSL certificate validation — High |

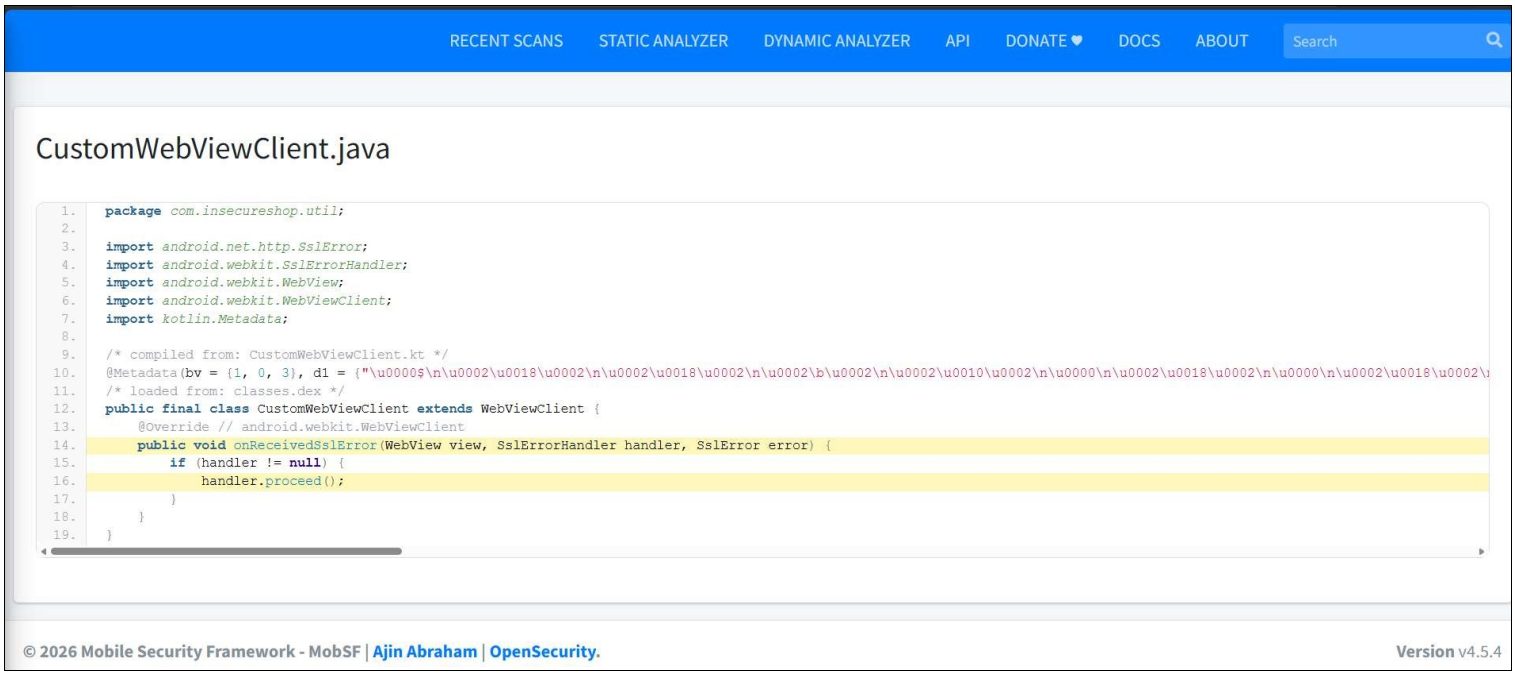
## Task 9 – MobSF Cross-Verification

**Objective:** To compare MobSF findings against manual evidence gathered in Part A and mark each as verified, not verified, or a false positive.

| MobSF Finding | Manual Evidence | Status |
|---|---|---|
| Cleartext traffic enabled (`android:usesCleartextTraffic=true`) | `AndroidManifest.xml` contains android:usesCleartextTraffic="true" | **Verified** |
| Insecure WebView SSL certificate validation | `CustomWebViewClient.java` overrides onReceivedSslError() and calls handler.proceed() | **Verified** |

**Screenshot**

*Manual verification of CustomWebViewClient.java, confirming SSL errors are bypassed via handler.proceed().*



## Conclusion

The static analysis of InsecureShop.apk showed that its most significant security-relevant observation is the combination of cleartext network traffic and an insecure WebView SSL implementation. MobSF flagged `android:usesCleartextTraffic="true"` as High severity, and this was manually confirmed in the decoded AndroidManifest.xml. More critically, `CustomWebViewClient.java` overrides `onReceivedSslError()` and unconditionally calls `handler.proceed()`, meaning the app accepts any SSL certificate, including invalid or attacker-controlled ones. Combined with cleartext traffic being permitted and a WebView that loads URLs supplied through an external Intent, this creates a realistic man-in-the-middle risk: an attacker on the same network could intercept or inject traffic loaded inside the WebView. Additional supporting observations, such as the password being written to Android logs in LoginActivity and the exported InsecureShopProvider returning stored credentials, further indicate weak handling of sensitive data. Both key MobSF alerts were manually verified against source and manifest evidence rather than accepted at face value, in line with the assignment's verification requirement.